project_2.pdf
PDF

how do i use the sklearn.tree.decisiontreeclassifier when i want to change the criterion, splitter, and max_depth in the function

Got it — your project (CS 6375 Machine Learning Project 2) asks you to **train a** `DecisionTreeClassifier` **and tune parameters like** `criterion`, `splitter`, **and** `max_depth` using scikit-learn.

Here's how you can use these parameters in code:

✅ Example Code

python Copy code

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score
from sklearn.model_selection import train_test_split, GridSearchCV
import pandas as pd

# Example: load one dataset
data = pd.read_csv("train_c500_d100.csv")
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# Split into train/validation if needed
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_sta

# --- Define parameter grid to tune ---
param_grid = {
    'criterion': ['gini', 'entropy', 'log_loss'],  # criteria for split quality
    'splitter': ['best', 'random'],                # how to choose split at each
    'max_depth': [None, 5, 10, 20, 50]             # limit tree depth
}

# --- Initialize Decision Tree ---
dt = DecisionTreeClassifier(random_state=42)

# --- Use GridSearchCV to find best params ---
```

```python
# --- Initialize Decision Tree ---
dt = DecisionTreeClassifier(random_state=42)

# --- Use GridSearchCV to find best params ---
grid_search = GridSearchCV(dt, param_grid, cv=5, scoring='f1', n_jobs=-1)
grid_search.fit(X_train, y_train)

print("Best parameters:", grid_search.best_params_)

# --- Evaluate on validation or test set ---
y_pred = grid_search.predict(X_val)
print("Accuracy:", accuracy_score(y_val, y_pred))
print("F1 Score:", f1_score(y_val, y_pred))
```

Copy code

🧠 Parameter meanings:

- `criterion`:
  - `'gini'` → Gini impurity (default)
  - `'entropy'` → Information gain (Shannon entropy)
  - `'log_loss'` → Similar to entropy, but smoother for probabilistic outputs.
- `splitter`:
  - `'best'` → chooses best split deterministically
  - `'random'` → chooses a random best split among top candidates (adds randomness)
- `max_depth`:
  - Integer or `None` (unlimited depth).
  - Smaller values prevent overfitting; larger ones allow more complex trees.

🧩 After tuning

Once you find the best parameters (e.g., via `grid_search.best_params_`), **retrain on the combined training + validation data** and then **evaluate on the test set**, per your project instructions.

Example:

```python
best_dt = DecisionTreeClassifier(**grid_search.best_params_, random_state=42)
best_dt.fit(pd.concat([X_train, X_val]), pd.concat([y_train, y_val]))
```

Copy code

```
y_test_pred = best_dt.predict(X_test)
print("Final Test Accuracy:", accuracy_score(y_test, y_test_pred))
print("Final Test F1:", f1_score(y_test, y_test_pred))
```

Would you like me to show you a short reusable function that loops through *all your CSV datasets* (train/valid/test) and stores each model's best parameters + F1/accuracy in a results table for your report?

Here's how you can set those options when you build a `DecisionTreeClassifier` in scikit-learn and tune them:

python Copy code

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import accuracy_score, f1_score
import pandas as pd

# load data
df = pd.read_csv("train_c500_d100.csv")
X, y = df.iloc[:, :-1], df.iloc[:, -1]
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_sta

# parameter grid
param_grid = {
    "criterion": ["gini", "entropy", "log_loss"],
    "splitter": ["best", "random"],
    "max_depth": [None, 5, 10, 20, 50]
}

# base tree and grid search
dt = DecisionTreeClassifier(random_state=42)
search = GridSearchCV(dt, param_grid, scoring="f1", cv=5, n_jobs=-1)
search.fit(X_train, y_train)

print("Best parameters:", search.best_params_)

# evaluate on validation
y_pred = search.predict(X_val)
print("Accuracy:", accuracy_score(y_val, y_pred))
```

```python
# evaluate on validation
y_pred = search.predict(X_val)
print("Accuracy:", accuracy_score(y_val, y_pred))
print("F1:", f1_score(y_val, y_pred))
```

You can also set those options directly when you build a tree, e.g.

```python
dt = DecisionTreeClassifier(criterion="entropy",
                            splitter="random",
                            max_depth=10,
                            random_state=42)
dt.fit(X_train, y_train)
```

Then call `dt.predict(X_val)` and compute metrics.
After tuning, retrain the best model on the combined training + validation data and test it, as the project instructions describe.

how can i use scikitlearn to get the accuracy and the f1 score of these predictions

Great question — scikit-learn makes this really simple once you have your predictions.

Let's say you already have:

```python
y_true = [...]   # the actual labels (e.g., from your test set)
y_pred = [...]   # the predicted labels (from model.predict())
```

Then you can compute **accuracy** and **F1 score** using these functions:

✅ Basic example

```python
from sklearn.metrics import accuracy_score, f1_score
```

✅ Basic example

python Copy code

```python
from sklearn.metrics import accuracy_score, f1_score

# compute metrics
acc = accuracy_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)

print("Accuracy:", acc)
print("F1 Score:", f1)
```

🧩 Example using a Decision Tree

python Copy code

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score
from sklearn.model_selection import train_test_split
import pandas as pd

# load data
df = pd.read_csv("train_c500_d100.csv")
X, y = df.iloc[:, :-1], df.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_s

# train model
dt = DecisionTreeClassifier(criterion="entropy", splitter="best", max_depth=10, r
dt.fit(X_train, y_train)

# make predictions
y_pred = dt.predict(X_test)

# evaluate
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score: {f1:.4f}")
```

Notes

- For **binary classification** (like your CNF datasets with labels `0` and `1`), `f1_score` works fine as shown.
- If you later test on **multi-class** data (e.g. MNIST), use:

```python
f1 = f1_score(y_test, y_pred, average='macro')  # or 'weighted'
```

- You can also compute both at once:

```python
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

  which gives precision, recall, and F1 for each class.

Would you like me to show how to automatically compute and store accuracy + F1 for **all datasets** (c300–c1800) in one table for your report?

how to itterate over rows of pandas dataframe

Good question — iterating over rows in a pandas DataFrame can be done several ways depending on what you need.

Here's a quick rundown of the **main methods**, with examples 👇

✅ 1. `iterrows()` — most common (but not the fastest)

Returns each row as a `(index, Series)` pair.

```python
import pandas as pd

df = pd.DataFrame({
```

```python
df = pd.DataFrame({
    "name": ["Alice", "Bob", "Charlie"],
    "age": [25, 30, 35]
})

for index, row in df.iterrows():
    print(index, row["name"], row["age"])
```

Output:

Copy code

```
0 Alice 25
1 Bob 30
2 Charlie 35
```

Use this when you just need to read values or do light row-by-row logic.

⚡ 2. `itertuples()` — faster and memory-efficient

Returns each row as a *namedtuple* (attributes instead of keys).

python Copy code

```python
for row in df.itertuples(index=False):
    print(row.name, row.age)
```

Output:

nginx Copy code

```
Alice 25
Bob 30
Charlie 35
```

This is **faster** than `iterrows()` and preferred when you only need to read data.

⚙️ 3. `apply()` — best for transforming or computing new columns

Instead of looping manually, vectorize operations using `.apply()`:

↓

python Copy code

⚙️ 3. `apply()` — best for transforming or computing new columns

Instead of looping manually, vectorize operations using `.apply()`:

python Copy code

```python
df["age_plus_5"] = df.apply(lambda row: row["age"] + 5, axis=1)
```

This avoids Python-level loops and is much faster for larger DataFrames.

🚀 4. Pure vectorized operations (fastest)

If you can, avoid iteration completely:

python Copy code

```python
df["age_plus_5"] = df["age"] + 5
```

This uses pandas' internal NumPy operations — extremely fast.

Summary

Method	Speed	Use Case
`.iterrows()`	🐢 Slow	Simple per-row logic or printing
`.itertuples()`	⚡ Fast	Read-only access
`.apply()`	🚀 Faster	Transform or compute new columns
Vectorized ops	🧨 Fastest	Arithmetic or column-based logic